OnboardAI

An AI assisted employee onboarding platform. It verifies uploaded documents automatically, answers policy questions from a company's own handbook with citations, and predicts which new hires are at risk of stalling before anyone in HR notices.

Course: Introduction to AI and Machine Learning **Institution:** Department of Artificial Intelligence and Data Science, Amrita Vishwa Vidyapeetham, Delhi NCR, Faridabad **Subject Coordinator:** Dr. Sakshi Ahuja

Member	Roll number	Role
Kritika Sah	DL.AI.U4AID25031	Team Lead
Prisha Saini	DL.AI.U4AID25039	Tester
Rohith K Nair	DL.AI.U4AID25043	Developer
Pasham Srikar	DL.AI.U4AID25049	QA

Current status

The mid term review covers work up to data splitting. That work is complete.

Area	Status
Problem definition and objectives	Complete
Three datasets built and characterised	Complete
Exploratory data analysis, 10 figures	Complete
Preprocessing with before and after evidence	Complete
Stratified splitting with a leakage audit	Complete
Validation suite, 34 cases and 55 checks	Complete, all passing
Model training and evaluation	Next phase

A working application, four trained models, a retrieval system and a multi agent assistant also exist. These were built ahead of schedule and belong to the final review. They are not part of the mid term assessment and are not presented as if they were.

Quick start from a clean machine

1. Prerequisites

- Python 3.10 or newer
- Node.js 18 or newer
- Ollama if you want the chat assistant to run. It is not needed for any

of the data work.

2. Back end

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

On macOS or Linux use `source venv/bin/activate` instead.

3. Build the data and models

Run these from the `backend` directory with the environment active.

```
python seed.py
python -m ml.knowledge_seed
python -m ml.make_demo_docs
python -m ml.train_classifier
python -m ml.train_nlp
python -m ml.train_risk
```

4. Generate the analysis

```
python -m ml.data_report
python -m ml.eda
python -m pytest tests -q
python -m tests.make_test_report
```

5. Language model, optional

```
ollama pull qwen2.5:7b
```

Nothing else is required. There is no API key and no cost. If you prefer a cloud model, copy `backend/.env.example` to `backend/.env` and set `LLM_PROVIDER` to `gemini` or `groq` with the matching key. The application degrades gracefully if no model is reachable: the dashboard works normally and the chat says plainly that AI features are offline.

6. Run the application

Two terminals.

```
cd backend
venv\Scripts\activate
uvicorn main:app --port 8000
```

```
npm install
npm run dev
```

The interface is at `http://localhost:5173` and the API at `http://localhost:8000`.

Demonstration logins:

Username	Password	Who
`employee 1`	`@1`	Shreya, Nimbus Labs
`employee 2`	`@2`	Adithya, blocked by an expired certificate
`hr1`	`@hr1`	HR for Nimbus Labs
`hr2`	`@hr2`	HR for Veltrix Corp

Project structure

```
onboardai/
■■■ backend/
■   ■■■ ml/
■   ■   ■■■ synth.py              Document corpus generator
■   ■   ■■■ text_synth.py         Message corpus generator
■   ■   ■■■ risk_synth.py         Journey dataset generator
■   ■   ■■■ data_report.py        Dataset card, quality checks, split audit
■   ■   ■■■ eda.py                Ten figures with written observations
■   ■   ■■■ train_classifier.py   Document classifier training
■   ■   ■■■ train_nlp.py          Intent and sentiment training
■   ■   ■■■ train_risk.py         Risk model training with ablations
■   ■   ■■■ extract.py            PDF and image text extraction
■   ■   ■■■ classifier.py         Document classifier inference
■   ■   ■■■ fields.py             Date, name and identifier extraction
■   ■   ■■■ validate.py           Document validation rules
■   ■   ■■■ pipeline.py           End to end document analysis
■   ■   ■■■ nlp_models.py         Intent and sentiment inference
■   ■   ■■■ pii.py                Personal data redaction
■   ■   ■■■ rag.py                Policy retrieval, access controlled
■   ■   ■■■ risk_model.py         Risk scoring with explanations
■   ■   ■■■ artifacts/           Generated models, reports and figures
■   ■■■ agents/
■   ■   ■■■ specialists.py       Six specialist agents and their tools
■   ■   ■■■ chatbot.py           Routing and dispatch
■   ■■■ routes/                  FastAPI endpoints
■   ■■■ tests/                   34 test cases, 55 checks
■   ■■■ knowledge/               Policy corpus for retrieval
■   ■■■ demo_documents/          Seven PDFs named by expected outcome
■■■ src/                         React front end
■■■ docs/                        Deck, paper, role documents, guides
```

Documentation

Document	What it covers
`docs/PROJECT_GUIDE.md`	The whole project, explained non technically then technically
`docs/CODEBASE_GUIDE.md`	Every file, split into mid term scope and the rest
`docs/roles/Tester_Role_Documentation.docx`	Test strategy, iteration process, defect log
`docs/roles/QA_Role_Documentation.docx`	Quality criteria, verification results, register
`docs/paper/OnboardAI_MidTerm_Paper.docx`	IEEE format paper, with LaTeX source alongside
`docs/OnboardAI_MidTerm_Review.pptx`	Presentation, 16 slides plus a 5 slide appendix
`backend/ml/artifacts/data_report.md`	Generated dataset card and quality audit
`backend/ml/artifacts/eda/observations.md`	The ten figures with their observations
`backend/ml/artifacts/test_report.md`	Every test case with expected and actual results

Every document with a `.docx` also has a `.tex` beside it for Overleaf.

Datasets

All three are generated. Real onboarding documents are identity and financial records that cannot lawfully be collected for a student project, and real onboarding outcomes do not exist because the system has never been deployed.

Dataset	Samples	Features	Target	Classes
Onboarding documents	4,950	Text, vectorised	Document type	9
Employee messages	3,360	Text, vectorised	Intent and sentiment	8 and 3
Onboarding journeys	6,000	12 numeric	Stalled, binary	2

The generators are built to make the task realistically difficult. Documents are degraded across three scan quality tiers with cross class text bleed. Messages include genuinely ambiguous examples whose label cannot be recovered from the text. Journeys are a latent variable process with an explicitly unobserved component, so perfect prediction is impossible by construction.

This is a limitation, not a feature. Every reported figure measures consistency with our own generators rather than real world capability. Replacing the document corpus with a public one such as RVL-CDIP is the first item of future work.

Known limitations

Listed in order of how much they constrain our conclusions.

1. All datasets are synthetic, so no result describes performance on real documents.
2. Sentiment accuracy is very high because the generator marks sentiment with stock phrases.

This is a property of the data and must not be read as model capability.

1. Generated documents average under 400 characters, far shorter than a real certificate.
2. The message corpus contains 200 duplicate records, 70 of which appear in both folds. The

impact was measured and found to be negligible, but the defect is real and unfixed.

1. Three journey features correlate above 0.90 with one another, so the feature set is

redundant.

1. The web API has no authentication. Anyone who can reach the port can read any record.
2. The project is not under version control.

Items 6 and 7 sit outside the mid term scope but are recorded because they are real.

Command reference

From `backend`:

```
python -m ml.data_report          # dataset card, quality checks, split audit
python -m ml.eda                  # ten figures and observations
python -m pytest tests -q         # 55 checks, about twenty seconds
python -m tests.make_test_report  # report with actual results
python -m ml.train_classifier     # retrain document classifier
python -m ml.train_nlp            # retrain intent and sentiment
python -m ml.train_risk           # retrain risk model with ablations
python -m ml.knowledge_seed       # write the policy corpus
python -m ml.make_demo_docs       # generate seven demonstration PDFs
python seed.py                    # reset the database
```

From the project root:

```
python docs/build_midterm_deck.py # presentation
python docs/build_paper.py        # IEEE paper, docx and tex
python docs/build_role_docs.py    # Tester and QA documents
npm run build                     # production front end build
npm run lint                      # front end lint
```

Reproducibility

Every dataset uses a fixed random seed of 42, so regenerating produces identical data. This is asserted by test TC-15. Every figure and every number in our reports is produced by one of the scripts above and none are typed in by hand, so if the data changes the documents change with it. The full test suite runs offline in about twenty seconds.